Slide 1: Matrix Partitioning & Load Balancing

Speaker: Fuzail

The Problem

Compute **C = A × B** on a distributed memory system where matrices may be too large to fit in a single node's RAM.

N × N **A** × N × N **B** = N × N **C**

Approach : 1D Row Wise Decomposition

SPMD Approach

Single Program, Multiple Data. Each rank executes the same multiplication logic on its assigned row slice.

Load Balancing

If `n % size != 0`, rows cannot be evenly divided. Some ranks will naturally be assigned more work than others.

My Solution

Base rows: `n / size`

The first `remainder` processes receive exactly 1 extra row.

```
rows_per_proc = n / size + (i < remainder ? 1 : 0);
```

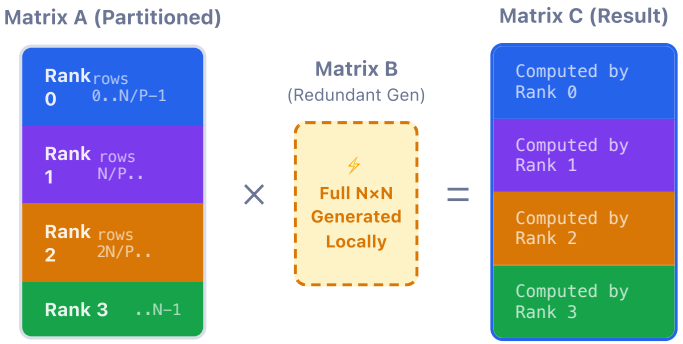
Why This Is Balanced

- **Negligible Imbalance:** Maximum difference between any two processes is at most 1 row. For large `n` (e.g., 8000), this is mathematically insignificant.
- **No Idle Time:** Ensures an incredibly even workload distribution.
- **Good Scalability:** Prevents faster nodes from idling at the final implicit barrier.

Partitioning Visualization (Zero-Comm)

Ranks: P=4

Matrix A is split row-wise across ranks. Matrix B is **fully generated locally on every rank** using redundant CPU cycles.



matmul.c — row distribution & displacement calculation

```c
for (int i = 0; i < size; i++) {
    int rows_per_proc = n / size + (i < remainder ? 1 : 0);
    sendcounts[i] = rows_per_proc * n;
    displs[i] = sum;

    if (i == rank) {
        my_rows = rows_per_proc;
        my_row_start = sum / n;  // Used to offset PRNG seed
    }
    sum += sendcounts[i];
}
```

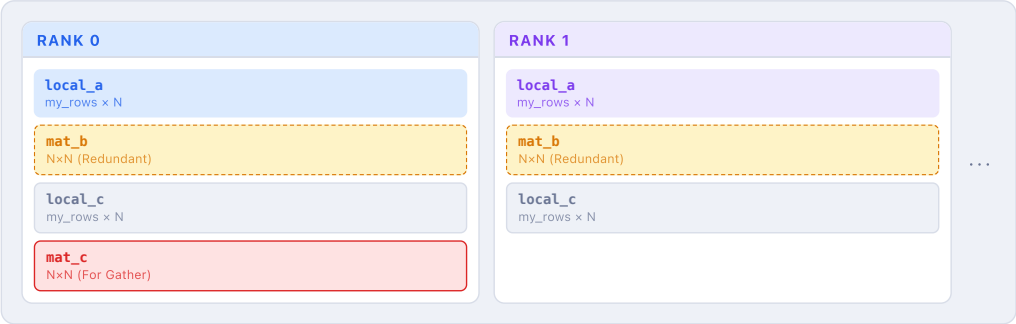
Slide 2: Memory Management& Initialization

Speaker: Md Fuzail

Streamlined Memory Map

local_a mat_b

No intermediate slices. Memory is allocated strictly for computation.



matmul.c — initialization

```c
/* EVERY rank generates full matrix B locally */
double *mat_b = alloc_matrix(n, n);
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        unsigned long state = concatenate(i, j) + seed + 1;
        mat_b[i * n + j] = my_rand(&state, 0, 1);
    }
}

/* Each rank generates its specific A rows using offset */
double *local_a = alloc_matrix(my_rows, n);
for (int i = 0; i < my_rows; i++) {
    int global_row_idx = my_row_start + i;
    for (int j = 0; j < n; j++) {
        unsigned long state = concatenate(global_row_idx, j) + seed + 0;
        local_a[i * n + j] = my_rand(&state, 0, 1);
    }
}
```

Data Structures

local_a — DISTRIBUTED

Only the slice needed is allocated: `my_rows × N`. Scales perfectly with node count.

mat_b (Full) — REDUNDANT GEN

No intermediate `slice` buffers. The full N×N matrix is generated straight into RAM using local CPU cycles.

local_c — RESULT BUFFER

Pre-initialized to zero using `memset`. Stores the local partial product of the multiplication.

1D Contiguous Allocation

All matrices use flat 1D arrays (`malloc(rows * cols)`). This prevents memory fragmentation, eliminates pointer-chasing, and guarantees cache locality.

Zero-Comm Strategy

Network Bypass

By abandoning collective communication completely, the cluster network bandwidth consumption is strictly **zero** during startup.

Offset Math

Nodes use `my_row_start + i` to independently seed the RNG. This guarantees all nodes compute the exact correct slice of Matrix A.

Constraint: Master Memory

Rank 0 must allocate a massive final `mat_c` (N×N) array just before `MPI_Gatherv` to receive and checksum the data.

MEMORY / NODE	INIT TIME
$O(N^2/P + N^2)$	$O(N^2)$
slice + full B replica	Fixed redundant CPU cost

Slide 3: Cache Optimization, Final Gather & Verification

Speaker C: Fuzail

⚡ i-k-j Compute Core

With data generated, isolated math begins immediately.

matmul.c — Cache Tiling

```
// Cache-friendly i-k-j loop order
for (int i = 0; i < my_rows; i++) {
    for (int k = 0; k < n; k++) {
        double a_val = local_a[i * n + k];
        for (int j = 0; j < n; j++) {
            local_c[i * n + j] +=
                a_val * mat_b[k * n + j];
        }
    }
}
```

🧠 **SPATIAL LOCALITY GUARANTEE**

By hoisting `a_val` and iterating over `j` internally, the hardware prefetcher reads `mat_b` linearly (stride-1). This drastically reduces L1/L2 cache misses compared to naive i-j-k loops.

📩 The Only Comm.

Network utilization remains strictly at **0%** until this exact line of code.

matmul.c — Final Gather

```
// Rank 0 allocates master buffer
double *mat_c = NULL;
if (rank == 0) mat_c = alloc_matrix(n,n);

MPI_Gatherv(local_c, my_rows * n, MPI_DOUBL
            mat_c, sendcounts, displs,
            MPI_DOUBLE, 0, MPI_COMM_WORLD);
```

SENDING LOCAL_C SLICES

Rank 0	Rank 1	Rank 2	Rank 3
0..N/P	N/P..	2N/P..	..N-1

↓ Gatherv ↓

🗄 Rank 0 (Master) — mat_c

R0 | R1 | R2 | R3

✅ 100% Deterministic

Once Gathered, Rank 0 verifies mathematical integrity against the baseline.

matmul.c — fmod Verify

```
if (rank == 0) {
    double checksum = 0.0;
    for (int i = 0; i < n * n; i++) {
        checksum += mat_c[i];
    }

    // Constrain checksum bounds
    checksum = fmod(checksum,
                    18446744073709551616.0);

    printf("Checksum: %f\n", checksum);
}
```

- **fmod Constraining**
  By applying a 64-bit floating-point modulo (`fmod`), we ensure massive results do not overflow, allowing exact 1:1 validation.

✓ Matches Assigned Constraints perfectly

Slide 4: Strategy Summary & Visualizations

Speaker : Fuzail

Hierarchical Architecture Overview

1. 1D Row Decomposition

A strict SPMD approach where the Matrix A workload is statically sliced horizontally across completely independent parallel worker nodes.

2. The Core Hypothesis

HPC networks are volatile. I hypothesized that burning local CPU cycles to generate data would bypass cluster network bottlenecks completely.

3. Memory Management

Avoided pointer arrays. All matrices strictly use 1D Contiguous Allocation (`malloc(rows * cols)`) to prevent memory fragmentation.

4. Perfect Load Balancing

Used deterministic offsets to cleanly distribute remainder rows to lower ranks. Maximum imbalance between any two nodes is strictly 1 row.

5. Cache Optimization

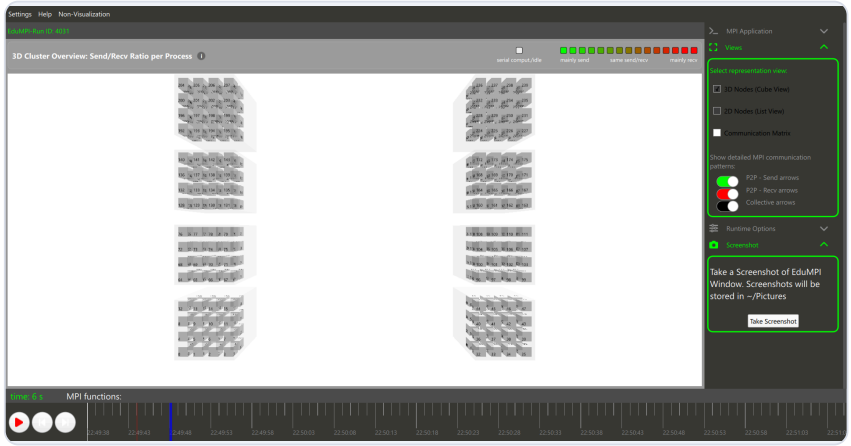
Refactored the compute loop to an `i-k-j` order. This enables linear stride-1 fetching of Matrix B, maximizing L1/L2 cache hits.

6. Zero-Comm Trade-off

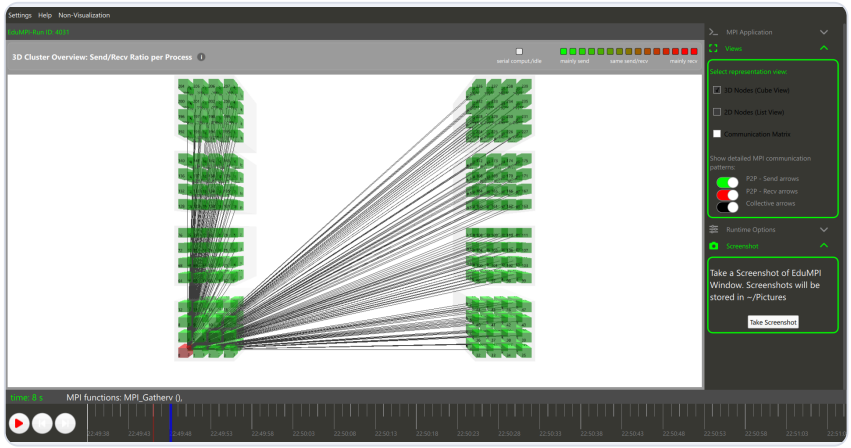
Achieved 0% network usage during setup by trading bandwidth for redundant `O(N²)` local PRNG generation of Matrix B on every node.

EduMPI Visualizations

Grey State: Isolated computation with 0% network utilization



Clean Gather: A single MPI_Gatherv operation at program exit



Slide 5: Architectural Comparison

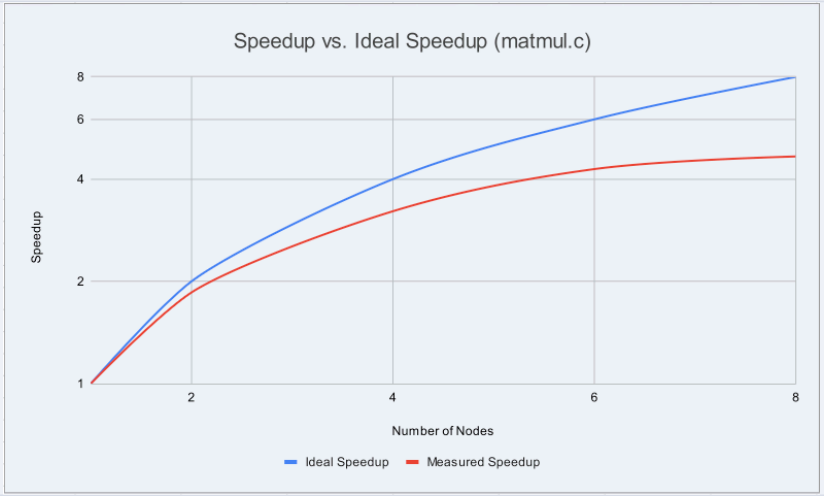
Speaker : Fuzail

Approach A MPI_ALLGATHERV

The Hypothesis: The CPU cost of generating 64 million random numbers via complex bitwise shifts is too heavy.

The Technical Implementation: Divide the PRNG generation workload proportionally across all ranks ($O(N^2/P)$). Once each node computes its slice, rely entirely on the high-speed cluster network (`MPI_Allgatherv`) to stitch the full 512 MB Matrix B together across all nodes.

Approach A Speedup Curve



Approach B ZERO-COMM

The Hypothesis: Transmitting 512 MB across a shared, multi-tenant university cluster will cause massive network congestion and unpredictable latency.

The Technical Implementation: Completely bypass the network. Accept a redundant computational penalty where every single CPU independently generates the entirety of Matrix B locally ($O(N^2)$). Achieve 0% network bandwidth consumption until the final verification gather.

Approach B Speedup Curve

